Week 3 Data Analytics: SQL Querying & Business Insights

AnalystLab Africa Data Analytics Internship Program

Date: June 18, 2026

Executive Summary

This document presents the SQL database setups, core/advanced queries, performance optimizations, and business insights for two databases:

1. **Chinook Database:** A media store database containing information about artists, albums, tracks, customers, and sales.

2. **Sales Database:** A transactional sales database representing retail orders, products, and customer details.

All queries have been successfully tested and executed on a local **MySQL** server.

1. Chinook Database (Music Store Data)

Schema & Entity-Relationship Analysis

The Chinook database schema represents a standard digital media store. Key relationships include:

- **Tracks & Catalog:** Artist has a one-to-many relationship with Album. Album has a one-to-many relationship with Track. Track is linked to Genre (one-to-many) and MediaType (one-to-many).
- **Customers & Employees:** Customer is supported by an Employee (represented by SupportRepId), and employees report to other employees (ReportsTo).
- **Sales Transactions:** Customer has a one-to-many relationship with Invoice. Invoice contains multiple InvoiceLine records, which reference the Track purchased (many-to-many junction table).

SQL Queries, Outputs & Insights

Query 1 (Core): Customer Filtering & Sorting

- **Objective:** Retrieve customers from USA, Canada, and Brazil, sorted by Country (ascending) and LastName (descending).
- **SQL Query:**

```
SELECT CustomerId, FirstName, LastName, Country, Email
  FROM Customer
  WHERE Country IN ('USA', 'Canada', 'Brazil')
  ORDER BY Country ASC, LastName DESC;
```

SELECT CustomerId, FirstName, LastName, Country, Email
  FROM Customer
  WHERE Country IN ('USA', 'Canada', 'Brazil')
  ORDER BY Country ASC, LastName DESC;

- **Business Insight:** Out of the 59 global customers, 26 reside in these three countries (USA: 13, Canada: 8, Brazil: 5). This makes the Americas the primary customer cluster for the Chinook store, suggesting localized marketing campaigns in these regions would yield high engagement.

Query 2 (Core): Aggregation, Grouping & Having

- **Objective:** Find billing cities with 7 or more invoices, including invoice count, total sales, and average invoice size.
- **SQL Query:**

```
SELECT BillingCity, BillingCountry,
       COUNT(InvoiceId) AS InvoiceCount,
       SUM(Total) AS TotalRevenue,
       ROUND(AVG(Total), 2) AS AverageInvoiceAmount
  FROM Invoice
  GROUP BY BillingCity, BillingCountry
  HAVING COUNT(InvoiceId) >= 7
  ORDER BY TotalRevenue DESC;
```

- **Sample Output:**

Prague	Czech Republic	14	90.24	6.45
Paris	France	14	77.24	5.52
Mountain View	USA	14	77.24	5.52
London	United Kingdom	14	75.24	5.37
São Paulo	Brazil	14	75.24	5.37
Berlin	Germany	14	75.24	5.37
Fort Worth	USA	7	47.62	6.80

- **Business Insight:** Cities like Prague, Paris, Mountain View, and London are top transaction volume hubs (14 invoices each). Interestingly, Prague has the highest total revenue ($90.24) and high average invoice values ($6.45), indicating that Czech customers tend to purchase larger bundles or higher-priced items.

Query 3 (Advanced): Multi-Table Joins

- **Objective:** Identify the top best-selling tracks including album, artist, and genre.
- **SQL Query:**

```
SELECT t.TrackId, t.Name AS TrackName, al.Title AS AlbumTitle, ar.Name AS ArtistName,
g.Name AS GenreName,
        COALESCE(SUM(il.Quantity), 0) AS TotalQuantitySold
 FROM Track t
 INNER JOIN Album al ON t.AlbumId = al.AlbumId
 INNER JOIN Artist ar ON al.ArtistId = ar.ArtistId
 INNER JOIN Genre g ON t.GenreId = g.GenreId
 LEFT JOIN InvoiceLine il ON t.TrackId = il.TrackId
 GROUP BY t.TrackId, t.Name, al.Title, ar.Name, g.Name
 ORDER BY TotalQuantitySold DESC, TrackName ASC
 LIMIT 10;
```

- **Business Insight:** Popular tracks across genres like Rock, Alternative, and Reggae (e.g., artists like Cidade Negra, Titãs, Gilberto Gil) represent steady sellers. No individual track has high single-unit dominance (max sales of 2 units in the sample), indicating that customers purchase a highly diversified catalog (long-tail distribution) rather than a few blockbusters.

Query 4 (Advanced): Subquery (Customer Lifetime Spending)

- **Objective:** Identify high-value customers who spent more than the average customer lifetime spending.
- **SQL Query:**

```
SELECT c.CustomerId, c.FirstName, c.LastName, c.Email, SUM(i.Total) AS TotalSpent
 FROM Customer c
 INNER JOIN Invoice i ON c.CustomerId = i.CustomerId
 GROUP BY c.CustomerId, c.FirstName, c.LastName, c.Email
 HAVING TotalSpent > (
    SELECT AVG(CustomerTotal.TotalSpent)
    FROM (
       SELECT SUM(Total) AS TotalSpent
       FROM Invoice
       GROUP BY CustomerId
    ) AS CustomerTotal
 )
 ORDER BY TotalSpent DESC;
```

- **Business Insight:** The average customer spends approximately $35.75 over their lifetime. 22 out of 59 customers exceed this benchmark. Helena Holý ($49.62) and Richard Cunningham ($47.62) lead. These customers should be enrolled in a premium loyalty program or sent exclusive offers to maximize retention.

Query 5 (Advanced): Window Functions (Genre Track Lengths)

- **Objective:** Rank tracks by length (milliseconds) within each genre and filter for the top 3 longest tracks per genre.
- **SQL Query:**

```
WITH TrackRanking AS (
    SELECT t.Name AS TrackName, g.Name AS GenreName, t.Milliseconds,
        DENSE_RANK() OVER (PARTITION BY t.GenreId ORDER BY t.Milliseconds
DESC) AS LengthRank
    FROM Track t
    INNER JOIN Genre g ON t.GenreId = g.GenreId
 )
 SELECT GenreName, TrackName, Milliseconds, LengthRank
 FROM TrackRanking
 WHERE LengthRank <= 3
 ORDER BY GenreName ASC, LengthRank ASC;
```

- **Business Insight:** This query highlights outliers in content length. For instance, the Rock genre's longest track is "Dazed And Confused" (1,612,329 ms or ~27 minutes). TV Shows and Sci-Fi/Fantasy genres dominate overall length, with episodes like "Occupation / Precipice" running over 1.47 hours (5,286,953 ms). This helps Chinook identify hosting/bandwidth requirements by understanding which genres contain very large media files.

Query 6 (Business): Top Performing Customers

- **Objective:** Top 5 customers by cumulative sales.
- **SQL Query:**

```
SELECT c.CustomerId, c.FirstName, c.LastName, c.Country,
       SUM(i.Total) AS TotalSpent,
       COUNT(i.InvoiceId) AS NumberOfInvoices
 FROM Customer c
 INNER JOIN Invoice i ON c.CustomerId = i.CustomerId
 GROUP BY c.CustomerId, c.FirstName, c.LastName, c.Country
 ORDER BY TotalSpent DESC
 LIMIT 5;
```

- **Outputs:**

1. Helena Holý (Czech Republic): $49.62 (7 invoices)
2. Richard Cunningham (USA): $47.62 (7 invoices)
3. Luis Rojas (Chile): $46.62 (7 invoices)
4. Ladislav Kovács (Hungary): $45.62 (7 invoices)
5. Hugh O'Reilly (Ireland): $45.62 (7 invoices)

Query 7 (Business): Revenue Trends Over Time

- **Objective:** Identify seasonal sales trends.
- **SQL Query:**

```
SELECT DATE_FORMAT(InvoiceDate, '%Y-%m') AS SalesMonth,
       SUM(Total) AS MonthlyRevenue,
       COUNT(InvoiceId) AS InvoicesCount
 FROM Invoice
 GROUP BY DATE_FORMAT(InvoiceDate, '%Y-%m')
 ORDER BY SalesMonth ASC;
```

- **Business Insight:** Monthly revenues consistently fluctuate around a baseline of $37.62, driven by a highly standardized billing cycle. The spikes (e.g., January 2022 at $52.62, April 2023 at $51.62) correspond to extra bulk purchases or special promotions.

Query 8 (Business): Customer Purchasing Behavior

- **Objective:** Investigate track purchase count, avg price, and unique genres purchased for top customers.
- **SQL Query:**

```
SELECT c.CustomerId, c.FirstName, c.LastName,
       COUNT(il.InvoiceLineId) AS TotalTracksPurchased,
       SUM(il.UnitPrice * il.Quantity) AS TotalTrackSpent,
       ROUND(AVG(il.UnitPrice), 2) AS AveragePricePerTrack,
       COUNT(DISTINCT t.GenreId) AS UniqueGenresPurchased
 FROM Customer c
 INNER JOIN Invoice i ON c.CustomerId = i.CustomerId
 INNER JOIN InvoiceLine il ON i.InvoiceId = il.InvoiceId
 INNER JOIN Track t ON il.TrackId = t.TrackId
 GROUP BY c.CustomerId, c.FirstName, c.LastName
 ORDER BY TotalTracksPurchased DESC
 LIMIT 10;
```

- **Business Insight:** The top customers purchase exactly 38 tracks. Interestingly, François Tremblay bought from 10 different genres, while Daan Peeters focused on only 4 genres. This reveals two consumer behaviors: **generalists** (who explore many genres) and **specialists** (who deep-dive into specific genres). Marketing strategies should send diverse recommendations to generalists and genre-specific promotions to specialists.

2. Sales Database (Retail Sales Data)

Schema & Entity-Relationship Analysis

The Sales database is loaded from a denormalized flat CSV file (sales_data_sample.csv) containing:

- **Orders:** Identified by ORDERNUMBER, ORDERLINENUMBER, and ORDERDATE.
- **Products:** Represented by PRODUCTCODE, PRODUCTLINE, MSRP, and PRICEEACH.
- **Customers:** Described by CUSTOMERNAME, PHONE, and geographic columns (CITY, STATE, COUNTRY, TERRITORY).
- **Deal Size:** Classified into Small, Medium, and Large based on transaction size.

SQL Queries, Outputs & Insights

Query 1 (Core): High-Value Classic Car Orders

- **Objective:** Find classic cars orders in 2003 or 2004 with a sales value > $5,000, sorted descending.
- **SQL Query:**

```
SELECT ORDERNUMBER, PRODUCTCODE, PRODUCTLINE, QUANTITYORDERED,
PRICEEACH, SALES, ORDERDATE
  FROM sales_data
  WHERE PRODUCTLINE = 'Classic Cars'
    AND YEAR_ID IN (2003, 2004)
    AND SALES > 5000
  ORDER BY SALES DESC;
```

- **Business Insight:** Classic cars represent a luxury catalog. Several orders exceed $10,000 in a single line (e.g., Order 10312 for product S10_1949 sold for $11,623.70 with 48 units). High quantity orders paired with a price cap of $100 per unit yield massive line values.

Query 2 (Core): Performance by Product Line

- **Objective:** Aggregate performance metrics (revenue, orders, transactions, qty, avg price) by Product Line.
- **SQL Query:**

```
SELECT PRODUCTLINE,
       COUNT(DISTINCT ORDERNUMBER) AS UniqueOrdersCount,
       COUNT(*) AS TotalTransactions,
       SUM(QUANTITYORDERED) AS TotalQuantitySold,
       ROUND(AVG(PRICEEACH), 2) AS AverageUnitPrice,
       ROUND(SUM(SALES), 2) AS TotalRevenue
  FROM sales_data
  GROUP BY PRODUCTLINE
  ORDER BY TotalRevenue DESC;
```

- **Outputs:**

Classic Cars	199	967	33,992	$87.34	$3,919,615.66
Vintage Cars	175	607	21,069	$78.15	$1,903,150.84
Motorcycles	72	331	11,663	$83.00	$1,166,388.34

Trucks and Buses	73	301	10,777	$87.53	$1,127,789.84
Planes	59	306	10,727	$81.74	$975,003.57
Ships	65	234	8,127	$83.86	$714,437.13
Trains	45	77	2,712	$75.65	$226,243.47

- **Business Insight: Classic Cars** is the undisputed anchor category, generating **$3.92M (39% of total revenue)**. **Vintage Cars** is second at **$1.90M**. Together, the automotive segment accounts for nearly 58% of the firm's total sales. Trains represent the smallest niche ($226K). Marketing and inventory spend should prioritize classic/vintage car inventory.

Query 3 (Advanced): Window Functions (Top Products per Category)

- **Objective:** Find the top 3 revenue-generating products within each product line.
- **SQL Query:**

```
WITH ProductSalesRanking AS (
    SELECT PRODUCTLINE, PRODUCTCODE,
        SUM(SALES) AS TotalSales,
        DENSE_RANK() OVER (PARTITION BY PRODUCTLINE ORDER BY
SUM(SALES) DESC) AS SalesRank
    FROM sales_data
    GROUP BY PRODUCTLINE, PRODUCTCODE
 )
 SELECT PRODUCTLINE, PRODUCTCODE, ROUND(TotalSales, 2) AS TotalSales,
SalesRank
 FROM ProductSalesRanking
 WHERE SalesRank <= 3
 ORDER BY PRODUCTLINE ASC, SalesRank ASC;
```

- **Business Insight:** Inside the Classic Cars line, product **S18_3232** is an absolute blockbuster, bringing in **$288,245.42** alone. For Motorcycles, **S10_4698** is the star ($170,401.07). Identifying these specific high-performing SKUs allows the supply chain team to prevent stockouts on key revenue drivers.

Query 4 (Advanced): Subqueries (High-Value Customers)

- **Objective:** Find customers whose average order value exceeds the average order value of all customers.
- **SQL Query:**

```
SELECT CUSTOMERNAME, COUNTRY,
       ROUND(AVG(OrderSales.TotalOrderSales), 2) AS CustomerAvgOrderValue,
       COUNT(DISTINCT OrderSales.ORDERNUMBER) AS TotalOrdersPlaced
 FROM (
    SELECT ORDERNUMBER, CUSTOMERNAME, COUNTRY, SUM(SALES) AS
TotalOrderSales
    FROM sales_data
    GROUP BY ORDERNUMBER, CUSTOMERNAME, COUNTRY
 ) AS OrderSales
 GROUP BY CUSTOMERNAME, COUNTRY
 HAVING CustomerAvgOrderValue > (
    SELECT AVG(AllOrders.TotalOrderSales)
    FROM (
       SELECT ORDERNUMBER, SUM(SALES) AS TotalOrderSales
       FROM sales_data
       GROUP BY ORDERNUMBER
    ) AS AllOrders
 )
 ORDER BY CustomerAvgOrderValue DESC;
```

- **Business Insight:** The overall average order value across all customers is **$32,638.39**. This query highlights the most valuable accounts. **Vida Sport, Ltd** (Switzerland) leads with an average order size of **$58,856.78**, followed by **AV Stores, Co.** (UK) at **$52,602.60**. These distributors place extremely high-value orders and should receive priority support and favorable payment terms.

Query 5 (Business): Top Performing Customers

- **Objective:** Top 10 customers by total sales contribution.
- **SQL Query:**

```
SELECT CUSTOMERNAME, COUNTRY,
       CONCAT(CONTACTFIRSTNAME, ' ', CONTACTLASTNAME) AS ContactName,
       ROUND(SUM(SALES), 2) AS TotalSpent
 FROM sales_data
 GROUP BY CUSTOMERNAME, COUNTRY, CONTACTFIRSTNAME,
CONTACTLASTNAME
 ORDER BY TotalSpent DESC
 LIMIT 10;
```

- **Outputs:**

Customer Name	Country	Contact Name	Total Sales ($)
Euro Shopping Channel	Spain	Diego Freyre	912,294.11
Mini Gifts Distributors Ltd.	USA	Valarie Nelson	654,858.06
Australian Collectors, Co.	Australia	Peter Ferguson	200,995.41
Muscle Machine Inc	USA	Jeff Young	197,736.94
La Rochelle Gifts	France	Janine Labrune	180,124.90

- **Business Insight: Euro Shopping Channel** and **Mini Gifts Distributors** are major outlier accounts, together generating over **$1.56M (15.6% of all revenue)**. A dedicated Account Manager should be assigned to protect these critical relationships.

Query 6 (Business): Quarterly Revenue & Trends

- **Objective:** Track growth and cumulative sales trends chronologically.
- **SQL Query:**

```
SELECT YEAR_ID, QTR_ID,
       ROUND(SUM(SALES), 2) AS QuarterlyRevenue,
       ROUND(SUM(SUM(SALES)) OVER (ORDER BY YEAR_ID, QTR_ID), 2) AS
CumulativeRevenue,
       COUNT(DISTINCT ORDERNUMBER) AS TotalOrders
 FROM sales_data
 GROUP BY YEAR_ID, QTR_ID
 ORDER BY YEAR_ID, QTR_ID;
```

- **Business Insight:** A clear seasonal trend is visible: **Q4 is the blockbuster quarter** every year. In 2003, Q4 revenue was **$1.86M** (nearly triple any other quarter). In 2004, Q4 revenue peaked at **$2.01M**. This is classic holiday season ordering behavior. Logistics, staffing, and inventory loading must scale up significantly in Q3 to handle this predictable Q4 volume surge.

Query 7 (Business): Deal Size Profile Analysis

- **Objective:** Profile sales performance by Deal Size.
- **SQL Query:**

```
SELECT DEALSIZE,
       COUNT(*) AS TransactionCount,
       SUM(QUANTITYORDERED) AS TotalQuantitySold,
       ROUND(SUM(SALES), 2) AS TotalSales,
       ROUND(AVG(SALES), 2) AS AverageSalesPerTransaction,
       ROUND(AVG(QUANTITYORDERED), 1) AS AverageQuantityPerTransaction
 FROM sales_data
 GROUP BY DEALSIZE
 ORDER BY TotalSales DESC;
```

- **Outputs:**
- **Medium Deals:** 1,384 transactions | $6,087,432.24 revenue | Avg quantity: 37.9
- **Small Deals:** 1,282 transactions | $2,643,077.35 revenue | Avg quantity: 30.5
- **Large Deals:** 157 transactions | $1,302,119.26 revenue | Avg quantity: 47.2
- **Business Insight:** Although **Large Deals** have the highest average value ($8,293.75), **Medium Deals** drive the vast majority of our business, accounting for **$6.09M (60.6% of total revenue)**. Small deals represent high administrative overhead (1,282 transactions) but only 26% of revenue. Optimization strategies should focus on pushing "Small" deal customers into "Medium" bundles.

3. Query Optimization & Indexing

To ensure these queries run efficiently on larger datasets, we analyzed execution plans using EXPLAIN.

Case Study: Group By Performance on `sales_data`

Let us examine the performance of grouping transactions by PRODUCTLINE (Query 2).

1. Before Indexing

- **Execution Plan (`EXPLAIN`):**

```
-> Table scan on <temporary>
    -> Aggregate using temporary table
       -> Table scan on sales_data  (cost=304 rows=2794)
```

- **Explanation:** Without an index, MySQL must perform a **Full Table Scan** (scanning all 2,823 rows). It then writes these rows to an on-disk **temporary table** to group and aggregate them. This process is slow, resource-heavy, and does not scale.

2. Optimization Applied

We created an index on the grouping column PRODUCTLINE:

```
CREATE INDEX idx_sales_productline ON sales_data(PRODUCTLINE);
```

3. After Indexing

- **Execution Plan (`EXPLAIN`):**

```
-> Group aggregate: sum(sales_data.SALES)  (cost=947 rows=7)
    -> Index scan on sales_data using idx_sales_productline  (cost=304 rows=2794)
```

- **Explanation:** After indexing, MySQL performs an **Index Scan** directly on idx_sales_productline. The database skips the temporary table generation entirely and executes a **Group Aggregate** on sorted index keys. This results in direct execution path improvements.

Recommended Production Indexes

We successfully applied the following indexes to target performance bottlenecks:

1. idx_sales_productline on sales_data(PRODUCTLINE) — Optimizes product line category aggregates.

2. idx_sales_customername on sales_data(CUSTOMERNAME) — Optimizes customer-level summaries.

3. idx_sales_ordernumber on sales_data(ORDERNUMBER) — Speed up subqueries and aggregations grouped by orders.

4. idx_sales_year_qtr on sales_data(YEAR_ID, QTR_ID) — Speeds up quarterly temporal reporting.

4. Professional Growth & LinkedIn Share

As part of the AnalystLab internship requirements, a post summarizing these key SQL learnings has been prepared for sharing on LinkedIn:

Conclusion

By importing flat files into structured SQL environments, building complex queries, and applying indexing strategies, we have established a robust pipeline for data-driven decision-making. The deliverables (chinook_queries.sql and sales_queries.sql) provide clean, reproducible scripts for production use.